

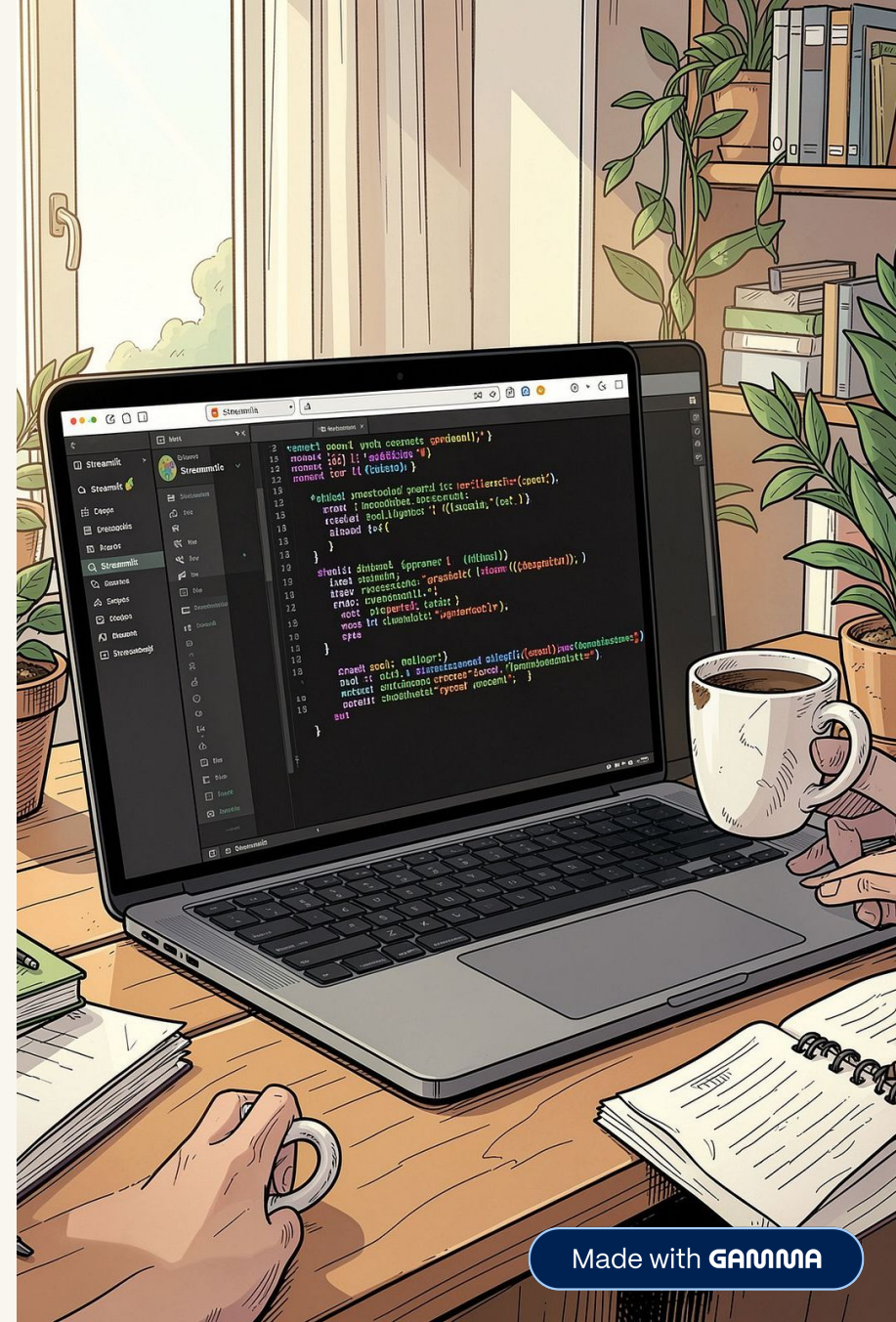
FinWise AI — Educational FinTech Prototype

A LangChain + Streamlit prototype for "Building LLM Applications with LangChain". Educational only: not connected to banks, does not execute transactions, and does not provide guaranteed financial advice. Consult a qualified financial professional before acting on recommendations.



One-line Project Summary

FinWise AI computes deterministic financial metrics in Python and complements them with streamed AI interpretations (JSON + plain-language insights) for explainable, demo-ready financial analysis.



Repository & Key Files



`app.py`

Streamlit UI: forms, dashboard tabs, and controls (run `streamlit run app.py`).



`src/prompts.py`

PromptTemplate & ChatPromptTemplate — builds structured JSON schema for LLM outputs.



`src/financial_calculator.py`

Deterministic maths: totals, ratios, and rule-based preliminary score (no AI).



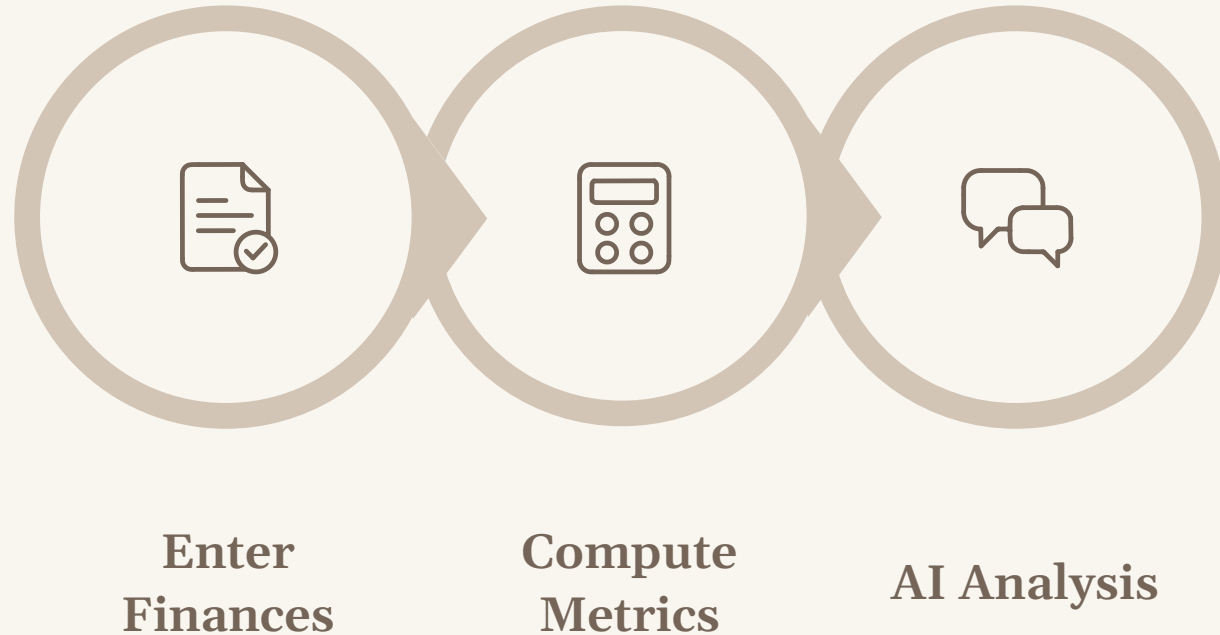
`src/cache_manager.py`

In-memory and SQLite caching interfaces for reducing redundant LLM calls.

1. Create & activate venv: `python -m venv .venv; source .venv/bin/activate`
(Windows: `.venv\Scripts\activate`)
2. Install: `pip install -r requirements.txt`
3. Configure API key: `cp .env.example .env` → edit `OPENAI_API_KEY`
4. Run app: `streamlit run app.py` (open `localhost:8501`)

- ❏ Keep the .env out of source control. This prototype is for demos and learning only.

Usage Flow — What the App Does



Users fill the form, review Python-verified metrics, then request AI Insights which returns a parsed JSON analysis and a streamed plain-language recommendation.

Deterministic vs Generative — Clear Separation

The app separates two layers:

Deterministic (Python)

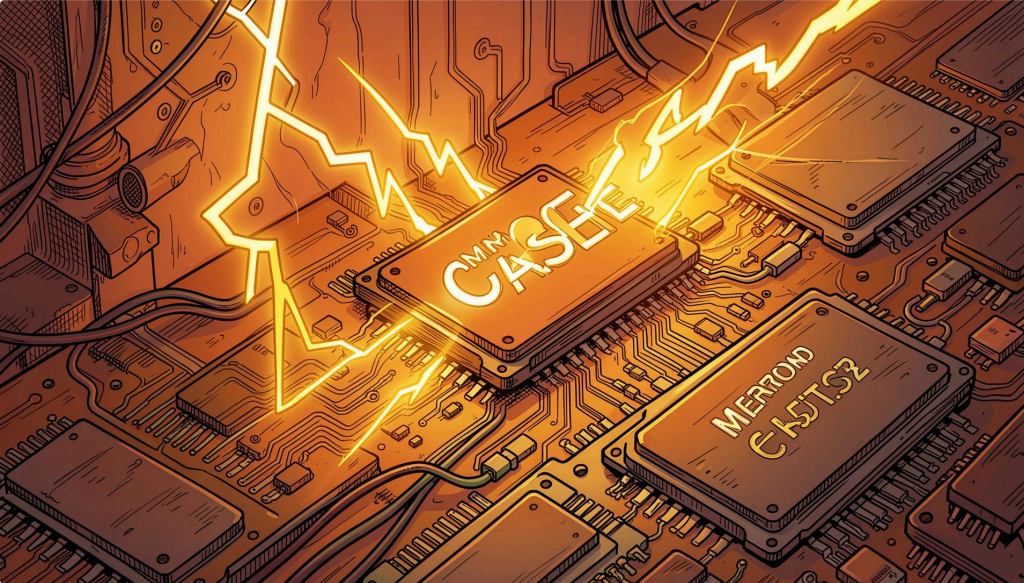
Total expenses, remaining income, savings ratio, expense ratios, debt ratio, and rule-based preliminary_score — reproducible, verifiable arithmetic.

Generative (LLM)

Uses ChatOpenAI via LangChain to interpret numbers into structured JSON (health score, risk level, priorities, tips) and a streamed narrative. Outputs can vary between runs.



Caching: InMemory vs SQLite



LangChain checks prompt + parameters; identical inputs return cached responses and avoid extra API calls, reducing cost during development.

InMemoryCache

Fastest, volatile — ideal for single-session demos. Does not survive restarts.

SQLiteCache

Persistent (.cache/finwise_cache.db). Slower than RAM but survives restarts — useful for repeated testing and demos.

Testing Scenarios & Expected Behaviour

Representative test inputs drive predictable Python outputs and illustrative AI responses:

- High-income low-expenses → positive remaining, high savings ratio, LOW risk; growth-focused tips.
- Income < expenses → negative remaining, HIGH risk; urgent cost-cutting priorities.
- High debt share → MEDIUM/HIGH risk; debt-reduction strategy recommended.
- Strong savings (~30%) → reinforcement of good habits, LOW risk.
- Zero remaining income → MEDIUM/HIGH risk; identify small savings opportunities.



Robustness & Safety Measures



- **Explicit Disclaimer**

The educational disclaimer appears in the sidebar and on relevant tabs; AI outputs are not financial advice.

- **Safe JSON handling**

All AI JSON is parsed with `src/utils.py::safe_parse_json` — malformed JSON falls back gracefully to avoid dashboard crashes.

- **Missing API key**

The UI surfaces a clear error if `OPENAI_API_KEY` is absent; the app will not crash.

Next Steps & Developer Notes

1. Harden prompts

Lock schema fields; add validation and guardrails to reduce hallucinations in JSON outputs.

2. Expand tests

Add unit tests for financial calculations and integration tests for caching behaviours and prompt stability.

3. Audit & logging

Record prompt hashes, cache hits, and model parameters for reproducibility during demos and evaluation.

4. UX & accessibility

Improve field labels, error states, and keyboard navigation; ensure colour contrasts meet accessibility standards.

Theme accents: #D3C5B6, #F2EBDF, #DECEBB. For questions or a live demo, open the repository's README and run streamlit locally.

